



디지털 입출력

디지털 입출력(I/O: Input/Output)

- 디지털 출력
 - High(1)/Low(0) 출력
→ 5[V]/0[V] 출력
- 디지털 입력
 - 5[V]/0[V] 입력
→ High(1)/Low(0) 입력
- 우노 보드 핀
 - 디지털 0 - 13번
 - 특별한 핀
 - ▶ 직렬 통신: 0(RX)번, 1(TX)번
 - ▶ PWM 출력: 3, 5, 6, 9, 10, 11번(~ 표시)
 - ▶ SPI 통신: 10, 11, 12, 13번



디지털 출력 제어 방법

- 디지털 출력 설정 함수

- mode: OUTPUT

```
void pinMode(uint8_t pin, uint8_t mode)
```

- 매개변수
 - pin : 설정하고자 하는 핀 번호
 - mode : INPUT, OUTPUT, INPUT_PULLUP 중 하나
- 반환값 : 없음

- 디지털 출력 함수

- HIGH(1)/LOW(0)
→ 5[V]/0[V]

```
void digitalWrite(uint8_t pin, uint8_t value)
```

- 매개변수
 - pin : 핀 번호
 - value : HIGH(1) 또는 LOW(0)
- 반환값 : 없음

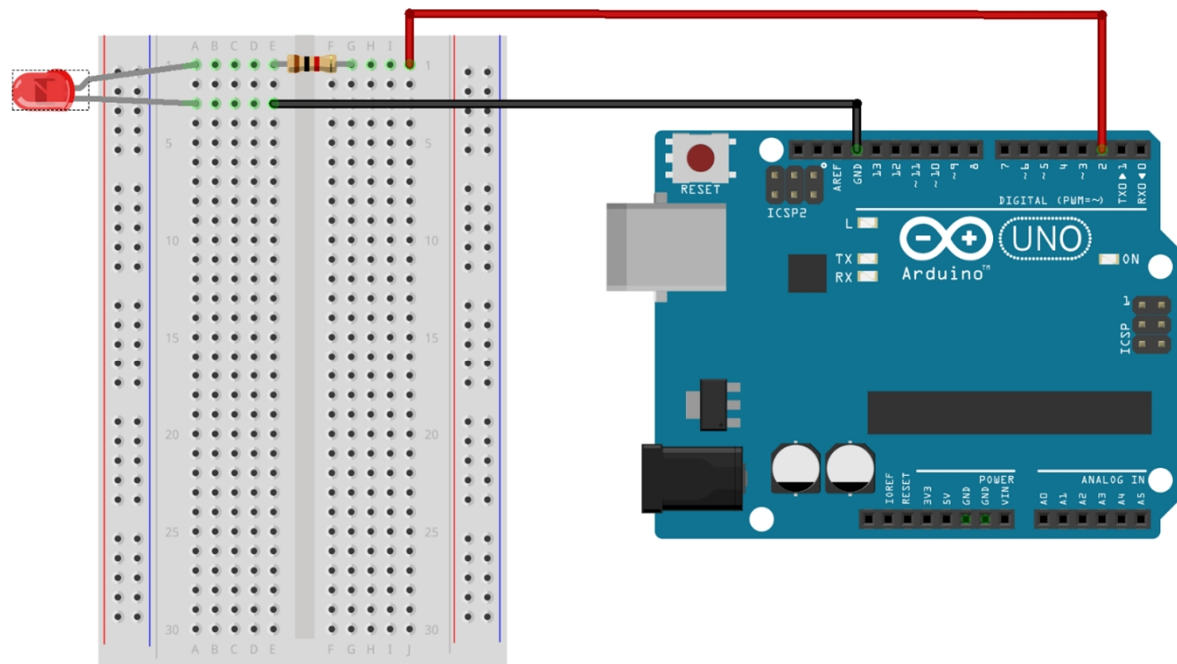
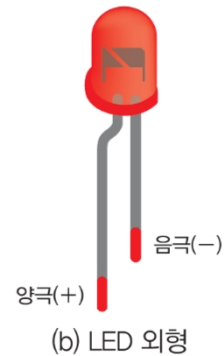
디지털 출력 사용 예

```
void setup() {  
  pinMode(13, OUTPUT);  
}  
  
void loop() {  
  digitalWrite(13, HIGH); // 13번 핀에 HIGH(1, 5[V]) 출력  
  delay(500);             // 약 500 msec 대기  
  digitalWrite(13, LOW);  // 13번 핀에 LOW(0, 0[V]) 출력  
  delay(500);             // 약 500 msec 대기  
}
```



LED on/off 제어 1

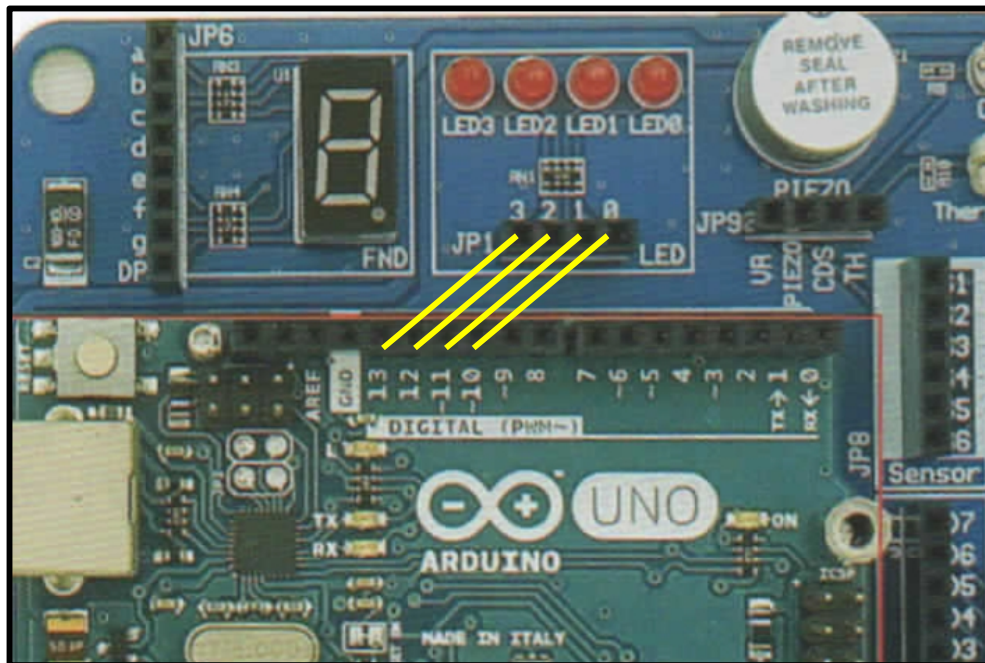
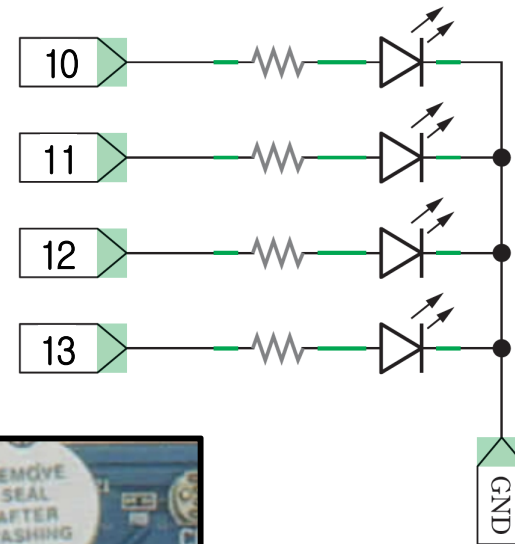
- LED(Light Emitting Diode)
 - 발광 다이오드
 - 양극에서 음극으로, 순방향으로만 전류가 흐름 → 발광



fritzing

LED on/off 제어 2

- 실습 회로 구성
 - LED0 → 디지털 10번
 - LED1 → 디지털 11번
 - LED2 → 디지털 12번
 - LED3 → 디지털 13번



LED on/off 제어 3

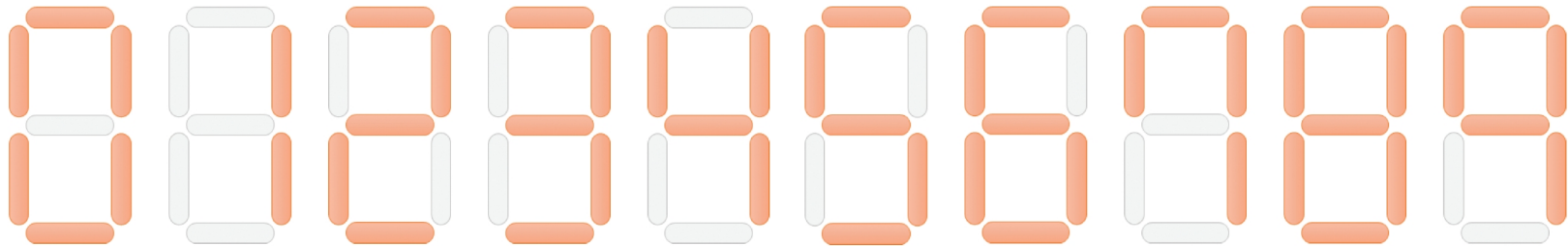
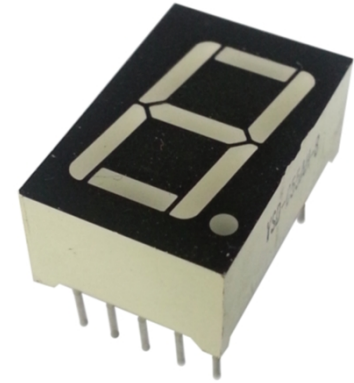
- LED3 on/off 제어
 - 500[msec] 동안 on
 - 500[msec] 동안 off
 - 무한 반복

```
void setup() {  
    pinMode(13, OUTPUT);  
}  
  
void loop() {  
    digitalWrite(13, HIGH); // 13번 핀에 HIGH(1, 5[V]) 출력  
    delay(500);             // 약 500 msec 대기  
    digitalWrite(13, LOW);  // 13번 핀에 LOW(0, 0[V]) 출력  
    delay(500);             // 약 500 msec 대기  
}
```

FND 표시 제어 1

- FND(Flexible Numeric Display)

- 숫자 표시용 소자
- 7개의 세그먼트(a, b, c, d, e, f, g)와
도트(p)로 구성
→ 7-segment라고도 불림

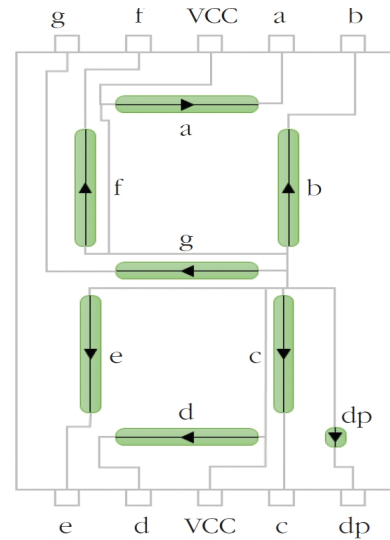
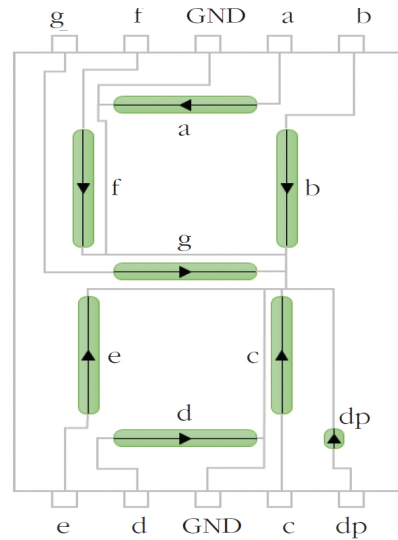
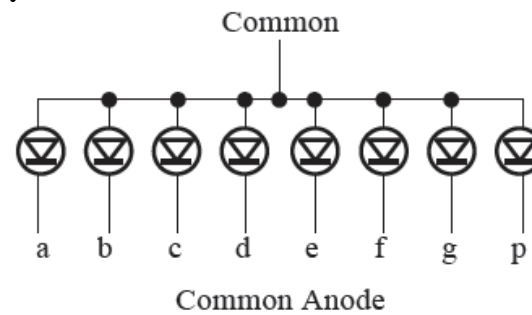
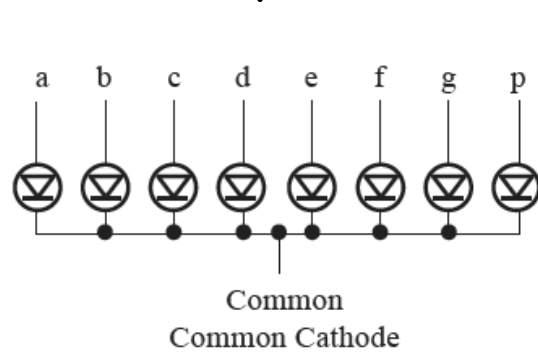


- 각 세그먼트는 LED로 구성

FND 표시 제어 2

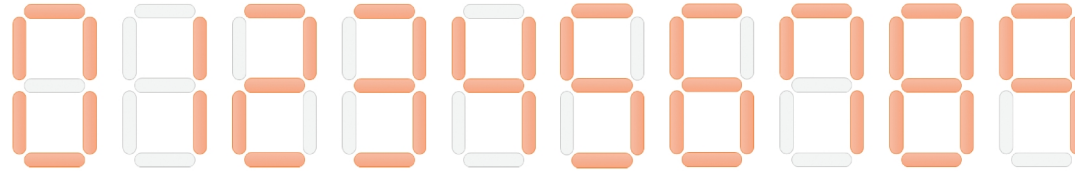
● FND 구조

- 공통 음극(common cathode)형 → 음극이 공통
- 공통 양극(common anode)형 → 양극이 공통



FND 표시 제어 3

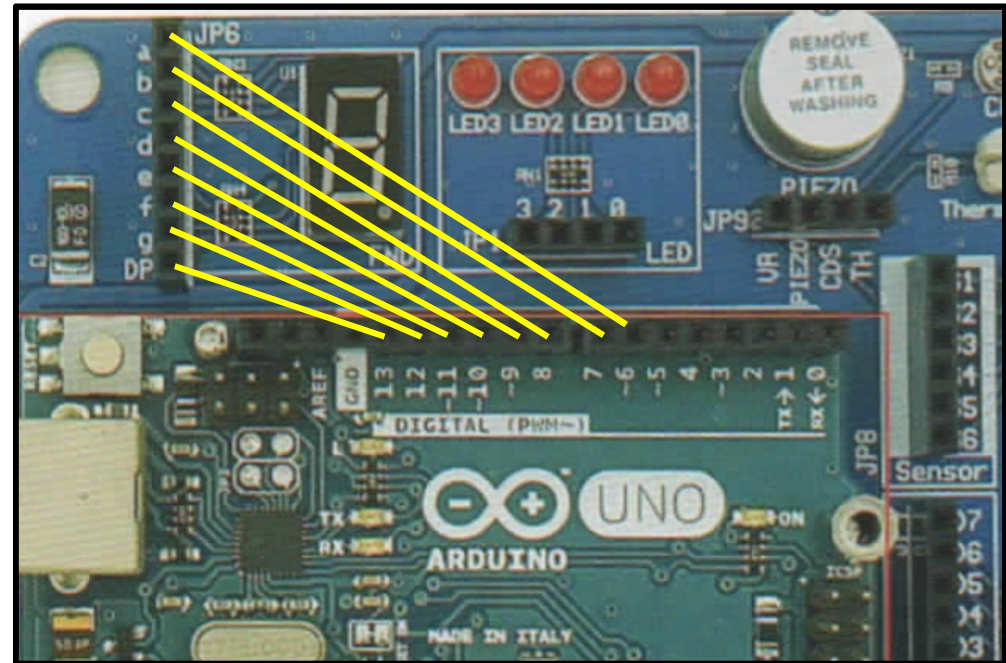
● 숫자 표시



숫자	세그먼트								제어값	
	dp	g	f	e	d	c	b	a	2진값	16진값
0	×	×	○	○	○	○	○	○	0011 1111	3F
1	×	×	×	×	×	○	○	×	0000 0110	06
2	×	○	×	○	○	×	○	○	0101 1011	5B
3	×	○	×	×	○	○	○	○	0100 1111	4F
4	×	○	○	×	×	○	○	×	0110 0110	66
5	×	○	○	×	○	○	×	○	0110 1101	6D
6	×	○	○	○	○	○	×	○	0111 1101	7D
7	×	×	○	×	×	○	○	○	0010 0111	27
8	×	○	○	○	○	○	○	○	0111 1111	7F
9	×	○	○	×	×	○	○	○	0110 0111	67

FND 표시 제어 4

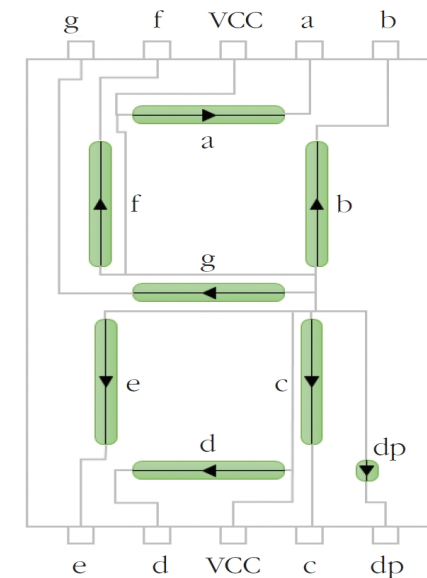
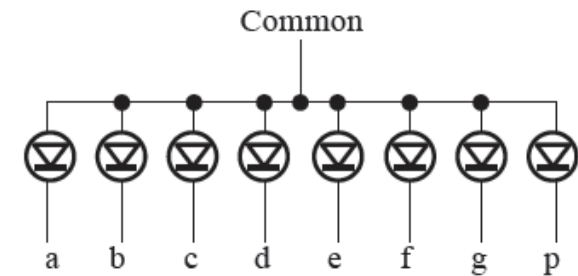
- 실습 회로 구성
 - FND a → 디지털 6번
 - FND b → 디지털 7번
 - FND c → 디지털 8번
 - FND d → 디지털 9번
 - FND e → 디지털 10번
 - FND f → 디지털 11번
 - FND g → 디지털 12번
 - FND DP → 디지털 13번



FND 표시 제어 5

- 공통 양극(common anode) FND
- 숫자 '0' 표시
 - Low 출력 → LED on
 - High 출력 → LED off

```
void setup() {  
    for (int i = 6; i <= 13; i++) {  
        pinMode(i, OUTPUT);  
    }  
}  
  
void loop() {  
    digitalWrite(6, LOW);  
    digitalWrite(7, LOW);  
    digitalWrite(8, LOW);  
    digitalWrite(9, LOW);  
    digitalWrite(10, LOW);  
    digitalWrite(11, LOW);  
    digitalWrite(12, HIGH);  
    digitalWrite(13, HIGH);  
}
```



FND 표시 제어 6

● 배열 활용

■ unsigned int fnd[] = {0x3F, 0x06, 0x5B, 0x4F, 0x66,
0x6D, 0x7D, 0x27, 0x7F, 0x67};

숫자	세그먼트								제어값	
	dp	g	f	e	d	c	b	a	2진값	16진값
0	×	×	○	○	○	○	○	○	0011 1111	3F
1	×	×	×	×	×	○	○	×	0000 0110	06
2	×	○	×	○	○	×	○	○	0101 1011	5B
3	×	○	×	×	○	○	○	○	0100 1111	4F
4	×	○	○	×	×	○	○	×	0110 0110	66
5	×	○	○	×	○	○	×	○	0110 1101	6D
6	×	○	○	○	○	○	×	○	0111 1101	7D
7	×	×	○	×	×	○	○	○	0010 0111	27
8	×	○	○	○	○	○	○	○	0111 1111	7F
9	×	○	○	×	×	○	○	○	0110 0111	67

FND 표시 제어 7

■ bitRead(x, n) 함수

▶ x의 우측에서 n번째 비트 값을 반환

```
const unsigned int fnd[] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x27, 0x7F, 0x67};
const int seg_io[] = {6, 7, 8, 9, 10, 11, 12, 13}; // a -> 6, b -> 7, ..., DP -> 13

void setup() {
    for (int i = 0; i <= 7; i++)
        pinMode(seg_io[i], OUTPUT);
}

void disp_fnd(unsigned int iDispNum)
{
    for (int i = 7; i >= 0; i--) {
        if (bitRead(fnd[iDispNum], i))
            digitalWrite(seg_io[i], LOW); // On
        else
            digitalWrite(seg_io[i], HIGH); // Off
    }
}

void loop() {
    for (int iNum = 0; iNum <= 9; iNum++) {
        disp_fnd(iNum);
        delay(500);
    }
}
```

디지털 입력 감지 방법 1



- 디지털 입력 설정 함수

- mode: INPUT

- 디지털 입력 함수

- 5[V]/0[V] → HIGH/LOW

void pinMode(uint8_t pin, uint8_t mode)

- 매개변수
 - pin : 설정하고자 하는 핀 번호
 - mode : INPUT, OUTPUT, INPUT_PULLUP 중 하나
- 반환값 : 없음

int digitalRead(uint8_t pin)

- 매개변수
 - pin : 핀 번호
- 반환값 : HIGH(1) 또는 LOW(0)

디지털 입력 감지 방법 2

```
const int SW0 = 2;
const int LED0 = 10;

void setup() {
  pinMode(SW0, INPUT);
  pinMode(LED0, OUTPUT);
}

void loop() {
  if(HIGH == digitalRead(SW0) ) { // 5[V] 입력
    digitalWrite(LED0, HIGH);
  }
  if(digitalRead(SW0)) { // 5[V] 입력
    digitalWrite(LED0, HIGH);
  }

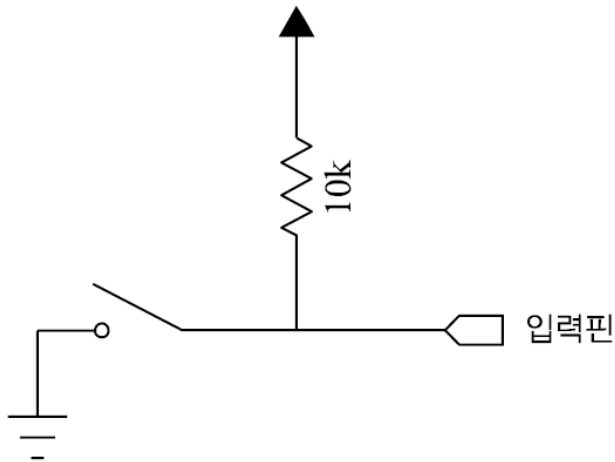
  if(LOW == digitalRead(SW0) ) { // 0[V] 입력
    digitalWrite(LED0, LOW);
  }
  if(!digitalRead(SW0) ) { // 0[V] 입력
    digitalWrite(LED0, LOW);
  }
}
```



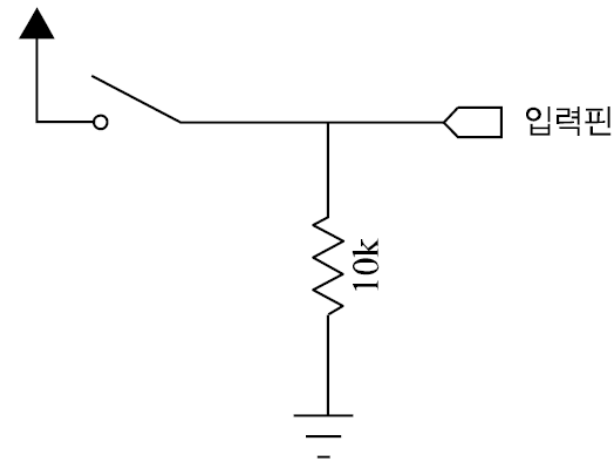
스위치 입력 처리 1

- 일반적인 스위치 입력 회로

- 스위치 on/off 상태에 따라 디지털 입력에 공급되는 전압이 변경되도록 회로 구성
- SW off $\rightarrow$ $V_{cc}[V]$ (HIGH)
- SW on $\rightarrow$ $0[V]$ (LOW)



- SW off $\rightarrow$ $0[V]$ (LOW)
- SW on $\rightarrow$ $V_{cc}[V]$ (HIGH)



- 스위치 on/off 상태 파악

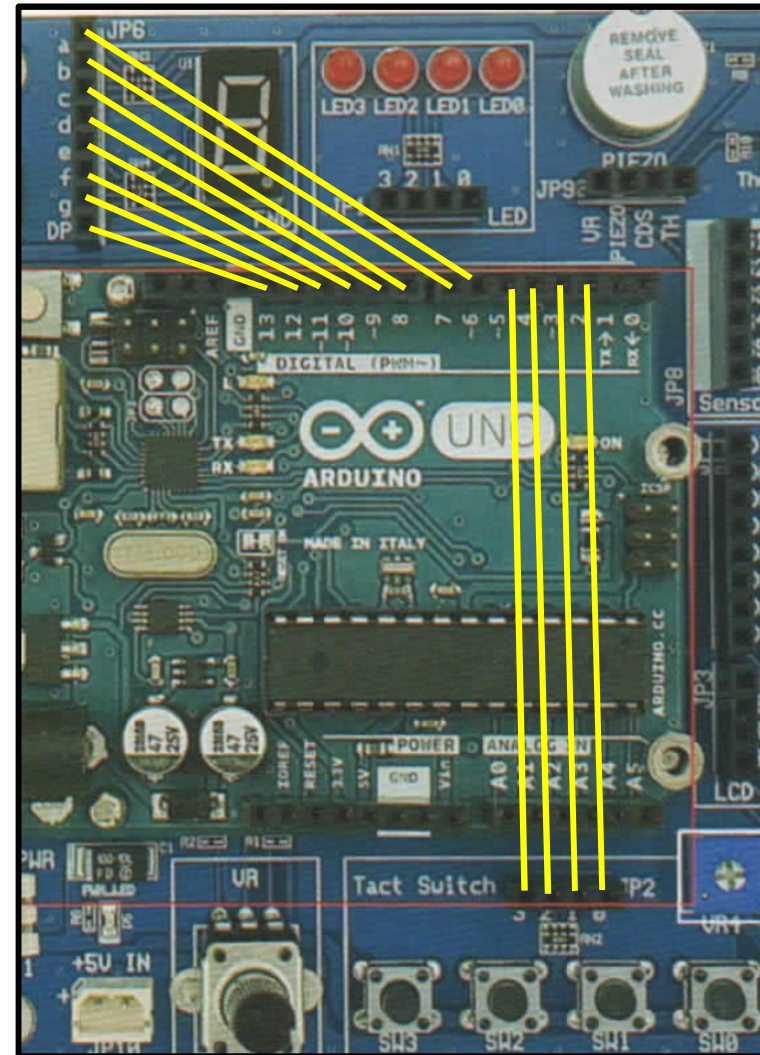
- 디지털 입력 값, HIGH/LOW로 파악

스위치 입력 처리 2

- 실습 회로 구성

- SW3 → 디지털 5번
- SW2 → 디지털 4번
- SW1 → 디지털 3번
- SW0 → 디지털 2번

- FND a → 디지털 6번
- FND b → 디지털 7번
- FND c → 디지털 8번
- FND d → 디지털 9번
- FND e → 디지털 10번
- FND f → 디지털 11번
- FND g → 디지털 12번
- FND DP → 디지털 13번



스위치 입력 처리 3

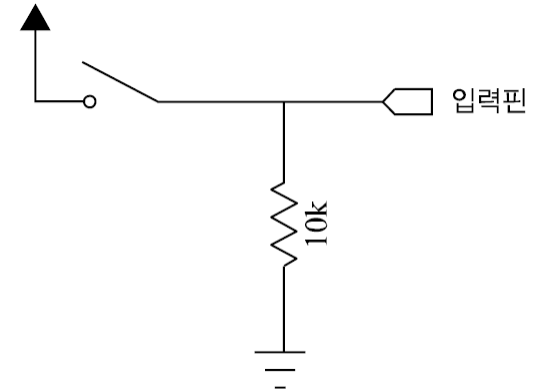
```
const int SWs[] = {2, 3, 4, 5};
const unsigned int fnd[] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x27, 0x7F, 0x67};
const int seg_io[] = {6, 7, 8, 9, 10, 11, 12, 13}; // a -> 6, b -> 7, ..., DP -> 13

void setup() {
    for (int i = 0; i <= 3; i++) pinMode(SWs[i], INPUT);
    for (int i = 0; i <= 7; i++) pinMode(seg_io[i], OUTPUT);
}

void disp_fnd(unsigned int iDispNum)
{
    for (int i = 7; i >= 0; i--) {
        if (bitRead(fnd[iDispNum], i))
            digitalWrite(seg_io[i], LOW); // On
        else
            digitalWrite(seg_io[i], HIGH); // Off
    }
}
```

스위치 입력 처리 4

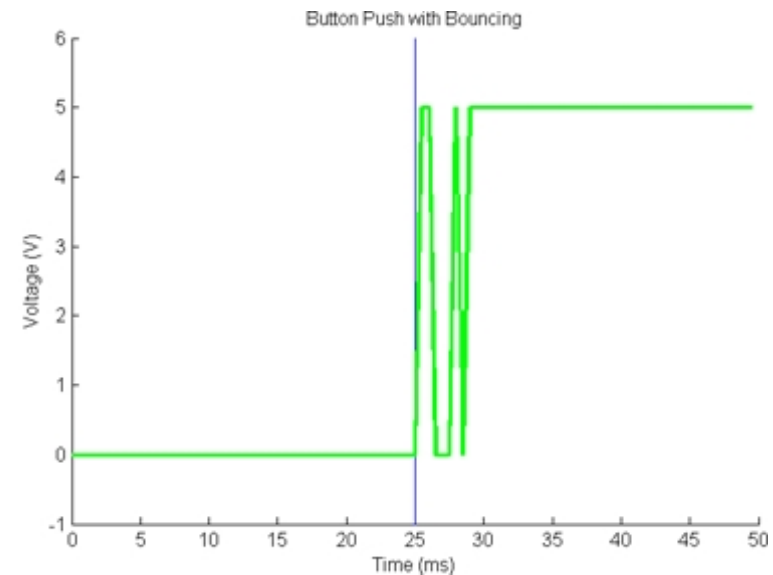
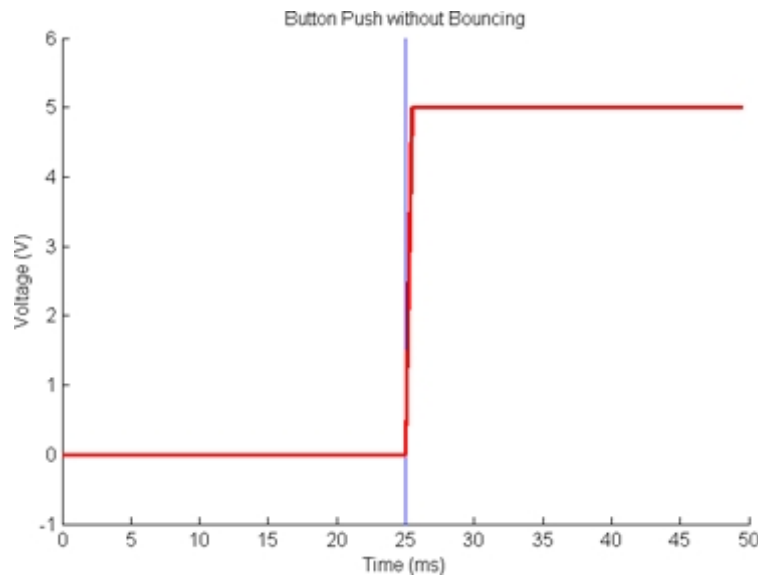
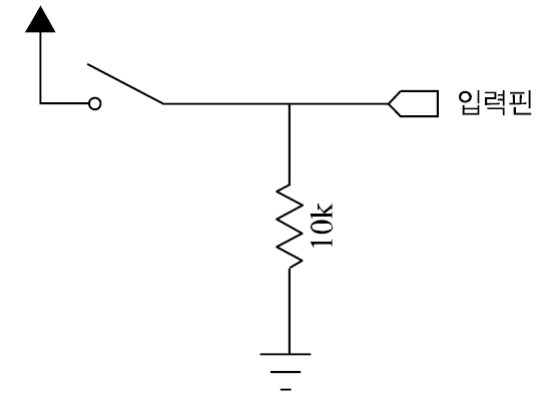
- 스위치 on → FND 숫자 표시
 - SW? on → FND 숫자 ? 표시 유지
- 실습 키트 스위치 상태와 입력 전압
 - SW on → Vcc[V](HIGH)
 - SW off → 0[V](LOW)



```
void loop() {  
    for(int i = 0; i <= 3; i++) {  
        if(digitalRead(SWs[i])) // SW i on  
            disp_fnd(i);  
    }  
}
```

스위치 바운싱(bouncing)

- 채터링(chattering)
- 버튼 스위치의 기계적 특성
 - 전압 변화 유발
 - 1회의 스위치 on을 2회 이상 발생한 것으로 오인 가능



소프트웨어 디바운싱(debouncing) 1

```
bool chkSw3On()  
{  
    static bool bPrevSw3On = false; // 스위치3의 과거 상태  
    // 스위치3의 on 여부  
    bool bSw3On = false;  
    // 스위치3의 현재 상태  
    bool bCrntSw3On = digitalRead(SWs[3]);  
  
    // 과거 상태가 off 이고 현재 상태가 on 이면  
    if(!bPrevSw3On && bCrntSw3On) {  
        delay(50);  
        // 50[msec] 이후의 스위치3의 현재 상태  
        bCrntSw3On = digitalRead(SWs[3]);  
        bSw3On = bCrntSw3On;  
    }  
  
    bPrevSw3On = bCrntSw3On;  
    return bSw3On;  
}
```

```
void loop() {  
    if(chkSw3On()) {  
        disp_fnd(3);  
        delay(300);  
    }  
    disp_fnd(0);  
}
```

소프트웨어 디바운싱(debouncing) 2

```
bool chkSwOn(int iSwNum)
{
    static bool bPrevSwOn[] = {false, false, false, false}; // 스위치의 과거 상태
    // 스위치의 on 여부
    bool bSwOn = false;
    // 스위치의 현재 상태
    bool bCrntSwOn = digitalRead(SWs[iSwNum]);

    // 과거 상태가 off 이고 현재 상태가 on 이면
    if(!bPrevSwOn[iSwNum] && bCrntSwOn) {
        delay(50);
        // 50[msec] 이후의 스위치의 현재 상태
        bCrntSwOn = digitalRead(SWs[iSwNum]);
        bSwOn = bCrntSwOn;
    }

    bPrevSwOn[iSwNum] = bCrntSwOn;
    return bSwOn;
}

void loop() {
    for(int i=0; i<4; i++) {
        if(chkSwOn(i)) {
            disp_fnd(i);
        }
    }
}
```